

```
In [1]: import pandas as pd
import numpy as np
```

```
In [2]: path = "../data/df_diabetic.csv"

df = pd.read_csv(path, low_memory=False)

df.head()
```

```
Out[2]:
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	dis
0	2278392	8222157	Caucasian	Female	[0-10)	NaN	6	
1	149190	55629189	Caucasian	Female	[10-20)	NaN	1	
2	64410	86047875	AfricanAmerican	Female	[20-30)	NaN	1	
3	500364	82442376	Caucasian	Male	[30-40)	NaN	1	
4	16680	42519267	Caucasian	Male	[40-50)	NaN	1	

5 rows × 49 columns

Feature Engineering

```
In [4]: df['severity'] = df['time_in_hospital'] * (df['number_diagnoses'] + 1)
```

```
In [5]: df['total_visits'] = (
    df['number_inpatient'] +
    df['number_outpatient'] +
    df['number_emergency']
)
```

Grouping ICD9 codes (diag_[1,2,3])

```
In [7]: def match_icd9_group(diag_col):
    #ICD9 codes referring to an external cause begin with a letter E-V. I'll replace
    diag_col = diag_col.str.replace(r'[E-V]\d*', '1000', regex = True).astype(float)

    #Matches each condition with its ICD9 grouping
    conditions = [
        (diag_col.between(390, 459)) | (diag_col == 785), #circulatory
        (diag_col.between(460, 519)) | (diag_col == 786), #respiratory
        (diag_col.between(520, 579)) | (diag_col == 787), #digestive
        np.floor(diag_col) == 250, #diabetes
    ]
```

```

diag_col.between(800, 999), #injury
diag_col.between(710, 739), #musculoskeletal
(diag_col.between(580, 629)) | (diag_col == 788), #genitourinary
diag_col.between(140, 239), #neoplasms (cancer/other tissue growths)
diag_col.isna() #na
] #anything else is defined as 'other'

groups = [
    'circulatory',
    'respiratory',
    'digestive',
    'diabetes',
    'injury',
    'musculoskeletal',
    'genitourinary',
    'neoplasms',
    'na'
]

return pd.Series(np.select(conditions, groups, default = 'other'))

```

```

In [8]: df['diag_1g'] = match_icd9_group(df['diag_1'])
df['diag_2g'] = match_icd9_group(df['diag_2'])
df['diag_3g'] = match_icd9_group(df['diag_3'])

df.drop(['diag_1', 'diag_2', 'diag_3'], axis=1, inplace=True)
df = df[df['diag_1g'] != 'na']
df = df[df['diag_2g'] != 'na']
df = df[df['diag_3g'] != 'na']

```

Grouping Rare Medical Specialties

```

In [10]: specialty_counts = df['medical_specialty'].value_counts()

rare_specialties = specialty_counts[
    specialty_counts < 500
].index

df['medical_specialty'] = df['medical_specialty'].replace(
    rare_specialties,
    'Other'
)

```

```

In [11]: df['medical_specialty'].dropna().unique()

```

```
Out[11]: <StringArray>
[
    'InternalMedicine',      'Family/GeneralPractice',
      'Cardiology',          'Surgery-General',
      'Orthopedics',          'Gastroenterology',
'Surgery-Cardiovascular/Thoracic',      'Nephrology',
  'Orthopedics-Reconstructive',          'Psychiatry',
      'Emergency/Trauma',          'Pulmonology',
      'Other',                  'ObstetricsandGynecology',
      'Urology',                'Surgery-Vascular',
      'Radiologist']
Length: 17, dtype: str
```

```
In [12]: specialty_map = {

    # Internal Medicine
    'InternalMedicine': 'Internal_Medicine',
    'Nephrology': 'Internal_Medicine',
    'Pulmonology': 'Internal_Medicine',
    'Gastroenterology': 'Internal_Medicine',
    'Cardiology': 'Internal_Medicine',

    # general
    'Family/GeneralPractice': 'General',

    # emergency
    'Emergency/Trauma': 'Emergency',

    # surgery
    'Surgery-Cardiovascular/Thoracic': 'Surgery',
    'Surgery-General': 'Surgery',
    'Orthopedics': 'Surgery',
    'Surgery-Vascular': 'Surgery',
    'Urology': 'Surgery',
    'Orthopedics-Reconstructive': 'Surgery',

    # psych
    'Psychiatry': 'Psych',

    # Reproductive
    'ObstetricsandGynecology': 'OB/GYN',

    # diagnosis
    'Radiologist': 'Diagnosis',

    #other
    'Other': 'Other',
}

df['specialty_grouped'] = df['medical_specialty'].map(specialty_map)
df['specialty_grouped'] = df['specialty_grouped'].fillna('Other')
df.drop('medical_specialty', axis=1, inplace=True)
```

Feature Selection

```
In [14]: corr = df.corr(numeric_only=True)["readmitted"].sort_values(ascending=False)
print(corr)
```

```
readmitted          1.000000
number_inpatient    0.163480
total_visits        0.124586
number_emergency    0.060433
discharge_disposition_id 0.050321
severity            0.048530
number_diagnoses    0.046176
time_in_hospital    0.043138
num_medications     0.036616
num_lab_procedures  0.019680
number_outpatient    0.018423
patient_nbr         0.006447
admission_source_id 0.005968
encounter_id        -0.010815
admission_type_id   -0.011089
num_procedures      -0.013108
Name: readmitted, dtype: float64
```

```
In [15]: feats_list = df.columns.to_list()
feats_list
```

```
Out[15]: ['encounter_id',
          'patient_nbr',
          'race',
          'gender',
          'age',
          'weight',
          'admission_type_id',
          'discharge_disposition_id',
          'admission_source_id',
          'time_in_hospital',
          'num_lab_procedures',
          'num_procedures',
          'num_medications',
          'number_outpatient',
          'number_emergency',
          'number_inpatient',
          'number_diagnoses',
          'max_glu_serum',
          'A1Cresult',
          'metformin',
          'repaglinide',
          'nateglinide',
          'chlorpropamide',
          'glimepiride',
          'acetohexamide',
          'glipizide',
          'glyburide',
          'tolbutamide',
          'pioglitazone',
          'rosiglitazone',
          'acarbose',
          'miglitol',
          'troglitazone',
          'tolazamide',
          'examide',
          'citoglipton',
          'insulin',
          'glyburide-metformin',
          'glipizide-metformin',
          'glimepiride-pioglitazone',
          'metformin-rosiglitazone',
          'metformin-pioglitazone',
          'change',
          'diabetesMed',
          'readmitted',
          'severity',
          'total_visits',
          'diag_1g',
          'diag_2g',
          'diag_3g',
          'specialty_grouped']
```

Feature Importance

Splitting

```
In [18]: from sklearn.model_selection import train_test_split

X = df.drop("readmitted", axis=1)
y = df["readmitted"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Pipeline

```
In [20]: from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
from imblearn.under_sampling import RandomUnderSampler

from sklearn.feature_selection import SelectFromModel
from sklearn.svm import LinearSVC
from sklearn.feature_selection import VarianceThreshold

In [21]: num_attribs = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_attribs = X.select_dtypes(include='object').columns.tolist()

num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_attribs),
    ('cat', cat_pipeline, cat_attribs)
])

# with svm selector
svm_selector = SelectFromModel(
    LinearSVC(C=0.01, random_state=631, max_iter=10000),
    threshold="median"
)

full_pipeline = Pipeline([
    ("preprocessing", preprocessor),
    ("feature_selection", svm_selector),
])
```

C:\Users\Matt\AppData\Local\Temp\ipykernel_25500\3003962619.py:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future version. Explicitly pass 'str' to 'include' to select them, or to 'exclude' to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

```
cat_attribs = X.select_dtypes(include='object').columns.tolist()
```

```
In [22]: X_train_prepared = full_pipeline.fit_transform(X_train, y_train)
X_test_prepared = full_pipeline.transform(X_test)

# undersampling for model comparison / feature selection
rus = RandomUnderSampler(replacement=False)
X_train_prepared, y_train = rus.fit_resample(X_train_prepared, y_train)

# variance threshold
# selector = VarianceThreshold(threshold=0.01)

# X_train_selected = selector.fit_transform(X_train_prepared)
# X_test_selected = selector.transform(X_test_prepared)
```

Baseline model for feature selection

```
In [24]: from sklearn.linear_model import LogisticRegression
import numpy as np

model = LogisticRegression(
    max_iter=1000,
    class_weight='balanced'
)

model.fit(X_train_prepared, y_train)

feature_names = full_pipeline.get_feature_names_out()

importance = pd.DataFrame({
    'Feature': feature_names,
    'Importance': np.abs(model.coef_[0])
})

importance = importance.sort_values(
    by='Importance',
    ascending=False
)
print("\nImportant Features:\n")
print(importance.head(20))
```

Important Features:

	Feature	Importance
77	cat__specialty_grouped_OB/GYN	0.784781
17	cat__weight_[125-150)	0.769857
12	cat__age_[10-20)	0.661253
16	cat__weight_[0-25)	0.612877
45	cat__rosiglitazone_Down	0.565710
33	cat__glimepiride_Down	0.486271
24	cat__metformin_Up	0.404835
32	cat__chlorpropamide_Steady	0.380867
47	cat__acarbose_Steady	0.376908
40	cat__glyburide_Down	0.373491
31	cat__chlorpropamide_No	0.332769
72	cat__diag_2g_neoplasms	0.306795
25	cat__repaglinide_Down	0.293842
4	num__number_inpatient	0.292016
42	cat__tolbutamide_Steady	0.279945
37	cat__glipizide_Down	0.248596
29	cat__nateglinide_Steady	0.247623
59	cat__glyburide-metformin_Up	0.247198
41	cat__tolbutamide_No	0.231848
53	cat__tolazamide_Steady	0.218092